

68. IndexedDB

Browser me built-in **database**, `localStorage` se kahin zyaada powerful.

- Lagbhag kisi bhi type ki values **key** ke saath store karta hai (kai key types).
- **Transactions** (reliability ke liye).
- **Key range queries, indexes.**
- `localStorage` se **bahut zyada** data.

Ye power aam client-server apps ke liye zyaada hai. IndexedDB **offline apps** ke liye hai (ServiceWorkers waghera ke saath).

Native interface **event-based** hai. `async/await` ke liye promise wrapper (jaise `idb`) hai, par pehle events se samjhte hain.

Data kahan? Aam taur par visitor ke home directory me, browser settings ke saath. Alag browsers aur OS users ka alag storage.

Database kholna

```
js
let openRequest = indexedDB.open(name, version);
```

- `name`: database ka naam
- `version`: positive integer (default `1`)

Ek origin me kai databases ho sakte hain. Alag websites ek doosre ke databases access nahi kar sakti.

`openRequest` par events:

- `success`: db ready, `openRequest.result` me database object
- `error`: khulne me fail
- `upgradeneeded`: db ready hai par uska version purana hai

Schema versioning built-in hai. Server-side databases me ye nahi hota. Yaha data browser me hai aur humare paas full-time access nahi, isliye nayi app version aane par user ke database ko update karna pad sakta hai. Local db version `open` me diye version se kam ho to `upgradeneeded` chalta hai. Ye tab bhi chalta hai jab db exist hi na kare (version `0`), to initialization bhi yahi hoti hai.

```
js
```

```

let openRequest = indexedDB.open("store", 1);

openRequest.onupgradeneeded = function() {
  // client ke paas db nahi tha: initialize karo
};
openRequest.onerror = function() {
  console.error("Error", openRequest.error);
};
openRequest.onsuccess = function() {
  let db = openRequest.result;
  // db ke saath kaam karo
};

```

Version 2 aane par:

```

js

let openRequest = indexedDB.open("store", 2);
openRequest.onupgradeneeded = function(event) {
  let db = openRequest.result;
  switch (event.oldVersion) {
    case 0: // db nahi tha: initialize
    case 1: // version 1 tha: update
  }
};

```

`onupgradeneeded` bina error ke khatam ho tabhi `onsuccess` chalta hai.

Database hatana: `indexedDB.deleteDatabase(name)`.

Purane version se `open` nahi kar sakte (db version 3, `open(...2)` error). Ye tab hota hai jab visitor ko purana JS code (proxy cache se) mil jaye. Bachne ke liye `db.version` check karke reload suggest karo, aur sahi HTTP caching headers do.

Parallel update problem

Ek tab me db version 1 khula hai, ab naya code aaya aur dusre tab me version 2 kholne ki koshish. Db dono tabs me shared hai, ek saath 1 aur 2 nahi ho sakta. Update ke liye version 1 ke saare connections band hone chahiye.

Isliye purane connection par `versionchange` event aata hai, use sun ke connection band karo. Nahi karoge to naya connection `blocked` event dega.

```

js

```

```

openRequest.onsuccess = function() {
  let db = openRequest.result;
  db.onversionchange = function() {
    db.close();
    alert("Database is outdated, please reload the page.");
  };
};

openRequest.onblocked = function() {
  // agar onversionchange sahi handle ho to ye nahi chalna chahiye
};

```

Kam se kam `onblocked` handler zaroor rakho, warna script chupchap mar jaati hai.

Object store

Data rakhne ki jagah. Baaki databases ke "tables/collections" jaisa. Ek db me kai stores (users, goods...).

- Lagbhag kuch bhi store ho sakta hai (complex objects bhi), **standard serialization algorithm** se (`JSON.stringify`) jaisa par zyaada types). Circular references wale objects nahi.
- **Har value ka unique `key`** hona chahiye. Key ka type: number, date, string, binary, ya array.

Store banana (synchronous, `await` nahi):

```

js
db.createObjectStore(name[, keyOptions]);

```

`keyOptions` (ek):

- `keyPath`: object ki property jo key banegi (jaise `id`)
- `autoIncrement`: `true` to key khud badhti hui number banti hai

Na do to store karte waqt key khud deni hogi.

```

js
db.createObjectStore('books', {keyPath: 'id'});

```

Store sirf `upgradeneeded` me ban/badal sakta hai (technical limitation). Bahar sirf data add/remove/update.

Upgrade ke 2 tarike: (1) per-version upgrade functions, ya (2)

`db.objectStoreNames.contains(name)` se dekhna ki kya hai:

```
js
openRequest.onupgradeneeded = function() {
  let db = openRequest.result;
  if (!db.objectStoreNames.contains('books')) {
    db.createObjectStore('books', {keyPath: 'id'});
  }
};
```

Store hatana: `db.deleteObjectStore('books')`.

Transactions

Operations ka group jo ya to **sab succeed** ya **sab fail**. (Jaise khareedne me: paisa kaatna + item add karna.)

IndexedDB me saare data operations transaction ke andar hone chahiye.

```
js
db.transaction(store[, type]);
```

- `store`: store ka naam (ya array of names)
- `type`:
 - `readonly` (default): sirf padhna
 - `readwrite`: padhna aur likhna (par stores banana/hatana nahi)

`versionchange` type bhi hai (sab kuch kar sakta hai) par manually nahi bana sakte, `upgradeneeded` ke liye automatic banta hai. Isliye stores yahi badalte hain.

Alag types kyun? Performance: kai `readonly` transactions ek hi store ko ek saath access kar sakte hain, par `readwrite` store ko **lock** kar deta hai.

```
js
```

```

let transaction = db.transaction("books", "readwrite"); // (1)
let books = transaction.objectStore("books"); // (2)

let book = { id: 'js', price: 10, created: new Date() };

let request = books.add(book); // (3)

request.onsuccess = function() { console.log("Book added", request.result); };
request.onerror = function() { console.log("Error", request.error); };

```

Value store karne ke 2 methods:

- `put(value, [key])`: same key ho to **replace**.
- `add(value, [key])`: same key ho to fail, `"ConstraintError"`.

`key` sirf tab dena hai jab store me `keyPath` ya `autoIncrement` na ho. `add` ka `request.result` = naye object ki key.

Transactions ka autocommit

Transaction ko "khatam" mark karne ka manual tarika (spec 2.0 me) **nahi** hai.

Jab transaction ki saari requests khatam ho jaayein aur microtask queue khaali ho, wo automatic commit ho jaata hai.

Side effect: transaction ke **beech me** `fetch`, `setTimeout` jaisa async kaam nahi daal sakte (ye macrotasks hain, transaction pehle hi commit ho chuka hoga):

```

js

request1.onsuccess = function() {
  fetch('/').then(response => {
    let request2 = books.add(anotherBook); // fail: TransactionInactiveError
  });
};

```

Transactions chhote (short-lived) hone chahiye (`readwrite` store ko lock karta hai, to doosre ko wait karna padta hai). Tarika: **pehle** `fetch` **waghera karke data taiyar karo, phir transaction banao aur saari db requests karo.**

Poori transaction ka safalta se save hona: `transaction.oncomplete` (sirf ye guarantee deta hai; individual requests success ho sakti hain par aakhri write fail ho sakta hai).

```

js

```

```
transaction.oncomplete = function() { console.log("Transaction is complete"); }  
transaction.abort(); // saare changes cancel, transaction.onabort
```

Error handling

Write requests fail ho sakti hain (jaise storage quota पूरी). **Fail hui request पूरी transaction ko abort kar deti hai** (saare changes cancel).

Kabhi kabhi handle karke transaction chalu rakhna ho to `request.onerror` me `event.preventDefault()`:

```
js  
  
request.onerror = function(event) {  
  if (request.error.name == "ConstraintError") {  
    console.log("Book with such id already exists");  
    event.preventDefault(); // transaction abort mat karo  
  }  
};  
  
transaction.onabort = function() { console.log("Error", transaction.error); };
```

Event delegation

IndexedDB events **bubble** karte hain: `request` -> `transaction` -> `database`. To `db.onerror` me saare errors pakad sakte ho:

```
js  
  
db.onerror = function(event) {  
  let request = event.target;  
  console.log("Error", request.error);  
};
```

Poori tarah handle hui error ko upar jaane se rokne ke liye `event.stopPropagation()`.

Searching

1. Key ya key range se
2. Kisi aur field se (iske liye index)

Key se

`IDBKeyRange` objects:

- `IDBKeyRange.lowerBound(lower, [open])`: `>= lower` (open ho to `> lower`)
- `IDBKeyRange.upperBound(upper, [open])`: `<= upper`

- `IDBKeyRange.bound(lower, upper, [lowerOpen], [upperOpen])`
- `IDBKeyRange.only(key)`

Methods (`query` = exact key ya range):

- `store.get(query)`: pehli value
- `store.getAll([query], [count])`
- `store.getKey(query)`
- `store.getAllKeys([query], [count])`
- `store.count([query])`

js

```
books.get('js')
books.getAll(IDBKeyRange.bound('css', 'html'))
books.getAll(IDBKeyRange.upperBound('html', true))
books.getAll()
books.getAllKeys(IDBKeyRange.lowerBound('js', true))
```

Store andar se **key ke hisaab se sorted** rehta hai, results bhi sorted aate hain.

Field se: Index

js

```
objectStore.createIndex(name, keyPath, [options]);
```

- `name`: index ka naam
- `keyPath`: jis field ko track karna hai
- `options`: `unique` (duplicate par error), `multiEntry` (value array ho to har member alag index key)

Index bhi `upgradeneeded` me banta hai:

js

```
openRequest.onupgradeneeded = function() {
  let books = db.createObjectStore('books', {keyPath: 'id'});
  let index = books.createIndex('price_idx', 'price');
};
```

Index har `price` value ke liye us price wale objects ki keys ki list rakhta hai, khud update hota rehta hai.

Search:

```
js

let transaction = db.transaction("books");
let books = transaction.objectStore("books");
let priceIndex = books.index("price_idx");

let request = priceIndex.getAll(10);
request.onsuccess = function() { console.log("Books", request.result); };

priceIndex.getAll(IDBKeyRange.upperBound(5)); // price <= 5
```

Index bhi tracked field ke hisaab se sorted hota hai.

Store se delete karna

```
js

books.delete('js'); // key/range se
books.clear();      // sab kuch
```

Field ke hisaab se delete: pehle index se key nikaalo, phir `delete`:

```
js

let request = priceIndex.getKey(5);
request.onsuccess = function() {
  books.delete(request.result);
};
```

Cursors

`getAll` sab kuch array me deta hai. Store memory se bada ho to fail. **Cursor** ek-ek key/value deta hai, memory bachata hai.

```
js

let request = store.openCursor(query, [direction]); // keys ke liye openKeyCu
```

`direction`:

- `"next"` (default): sabse chhoti key se upar
- `"prev"`: ulta

- `"nextunique"`, `"prevunique"`: same key wale records skip (sirf index cursors par)

`request.onsuccess` har result ke liye baar baar chalta hai.

```
js

let request = books.openCursor();
request.onsuccess = function() {
  let cursor = request.result;
  if (cursor) {
    console.log(cursor.key, cursor.value);
    cursor.continue();
  } else {
    console.log("No more books");
  }
};
```

Methods: `advance(count)` (skip karke aage), `continue([key])` (agli value par).

Index par cursor: `cursor.key` = index key (price), object ki key `cursor.primaryKey`:

```
js

let request = priceIdx.openCursor(IDBKeyRange.upperBound(5));
request.onsuccess = function() {
  let cursor = request.result;
  if (cursor) {
    console.log(cursor.key, cursor.primaryKey, cursor.value);
    cursor.continue();
  }
};
```

Promise wrapper (`idb`)

Har request par `onsuccess/onerror` lagana jhanjhat hai, to `async/await` wala wrapper (<https://github.com/jakearchibald/idb>):

```
js
```

```

let db = await idb.openDB('store', 1, db => {
  if (db.oldVersion == 0) {
    db.createObjectStore('books', {keyPath: 'id'});
  }
});

let transaction = db.transaction('books', 'readwrite');
let books = transaction.objectStore('books');

try {
  await books.add(...);
  await books.add(...);
  await transaction.complete;
  console.log('jsbook saved');
} catch (err) {
  console.log('error', err.message);
}

```

Ab `try..catch` chalta hai. Uncaught error "unhandled promise rejection" ban jaata hai:

```

js

window.addEventListener('unhandledrejection', event => {
  let request = event.target;      // native request object
  let error = event.reason;
});

```

"Inactive transaction" pitfall

Wrapper me bhi wahi: transaction ke beech me `fetch` jaisa macrotask daala to transaction auto-commit ho jaata hai aur agli request fail:

```

js

await inventory.add({...});
await fetch(...);                // (*)
await inventory.add({...});      // Error: inactive transaction

```

Solution: pehle data/fetch taiyar karo, phir db me save, ya nayi transaction banao.

Native objects

Kabhi original request chahiye ho to `promise.request`:

```

js

```

```
let promise = books.add(book);  
let request = promise.request;  
let transaction = request.transaction;
```

Summary

IndexedDB = "localStorage on steroids": simple key-value database, offline apps ke liye kaafi powerful. Basic usage:

1. Promise wrapper ((`idb`)) lo.
2. Database kholo: (`idb.openDb(name, version, onupgradeneeded)`); stores aur indexes (`onupgradeneeded`) me.
3. Requests ke liye transaction ((`readwrite`) chahiye to), phir (`transaction.objectStore('books')`).
4. Key se search seedhe store par; kisi field se search ke liye index.
5. Data memory me na aaye to cursor.